

Mentor AI — Local Model Training Guide

Step-by-step, reproducible guide for training the emotion classification model locally on a Lenovo LOQ with 6 GB VRAM

Core recommendation: Start with TF-IDF + Logistic Regression and Linear SVM. These models are CPU-friendly, fast, explainable, and fully suitable for the core FYP. Your 6 GB VRAM GPU is not required.

1. What You Are Building

Mentor AI will contain a supervised NLP classifier that reads a user's journal/check-in/message and predicts an emotional class. The prediction is passed to the existing generative Mentor AI layer as structured context.

- Input: user-generated text.
- Output: emotion label and, where supported, probability/confidence.
- Models: Logistic Regression baseline and Linear SVM comparison, both using TF-IDF.
- Deployment: Python/FastAPI model service called by the existing Node.js/Express backend.
- This is emotion classification, not medical or psychological diagnosis.

2. Hardware Expectations

Resource	Guidance
GPU	6 GB VRAM is sufficient for optional future transformer experiments; not required for the core classical pipeline.
RAM	16 GB is comfortable. 8 GB can work if the dataset is handled carefully.
CPU	A modern laptop CPU is sufficient.
Storage	Keep roughly 5–10 GB free for dataset, environment, notebooks, reports, and model artifacts.
OS	Windows 10/11 or Linux. Examples below use Windows commands where relevant.

3. Install Python and Create the Environment

Install a current stable Python 3.x release supported by your ML packages. On Windows, enable the PATH option during installation.

```
python --version
pip --version
```

Create an isolated virtual environment:

```
mkdir mentor-ai-ml
cd mentor-ai-ml
python -m venv .venv
# PowerShell
.\.venv\Scripts\Activate.ps1
# Command Prompt alternative:
# .venv\Scripts\activate.bat
```

Install the core packages:

```
python -m pip install --upgrade pip
pip install pandas numpy scikit-learn matplotlib seaborn jupyter joblib fastapi uvicorn datasets
pip freeze > requirements.txt
```

4. Project Structure

```
mentor-ai-ml/
├── .venv/
├── data/
│   ├── raw/
│   ├── processed/
│   └── README.md
├── notebooks/
│   ├── 01_dataset_exploration.ipynb
│   └── 02_model_experiments.ipynb
├── src/
│   ├── prepare_data.py
│   ├── train.py
│   ├── evaluate.py
│   └── predict.py
├── models/
├── reports/
│   ├── figures/
│   └── metrics/
├── app/
│   └── main.py
├── requirements.txt
└── README.md
```

5. Dataset: GoEmotions as the Starting Point

Use a public, appropriately licensed emotion-labeled text dataset. GoEmotions is a strong starting point because it provides fine-grained emotion annotations. Do not blindly use every label. First inspect the label set and define a smaller Mentor AI taxonomy.

Suggested initial classes: Joy, Sadness, Fear/Anxiety, Anger, Guilt, Frustration, Motivation, Neutral.

Some desired categories do not map one-to-one to the source dataset. Keep an explicit label-mapping document in the repository. If a class is not adequately represented, change the mapping or add appropriately labeled data rather than forcing weak labels.

6. Load and Inspect the Dataset

```
from datasets import load_dataset
```

```
dataset = load_dataset("google-research-datasets/go_emotions")
print(dataset)
print(dataset["train"][0])
```

- Check train/validation/test sizes.
- Inspect columns and label IDs/names.
- Count examples per label.
- Check missing/empty text.
- Check duplicate records.
- Read examples from every selected class.
- Look for noisy or ambiguous labels.
- Document all filtering and label-mapping decisions.

7. Preprocessing Rules

Keep preprocessing conservative. Removing too much information can damage emotion signals such as negation, punctuation, emojis, or informal wording.

- Remove or explicitly handle missing text.
- Remove exact duplicates where appropriate.
- Normalize repeated whitespace.
- Start with TF-IDF lowercase=True.
- Do not automatically remove stop words.
- Stemming/lemmatization is not required for the baseline.
- Consider normalizing URLs/usernames if they dominate the data.
- Do not automatically delete emojis.

8. Avoid Data Leakage

The vectorizer must learn its vocabulary and statistics from training text only. Never fit TF-IDF on the entire dataset before the final test evaluation.

```
X_train = train["text"]
y_train = train["label"]
X_test = test["text"]
y_test = test["label"]

from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(
    lowercase=True,
    ngram_range=(1, 2),
    min_df=2,
    max_df=0.95,
    sublinear_tf=True
)

X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)
```

9. Train Model 1 — Logistic Regression

```
from sklearn.linear_model import LogisticRegression
```

```
lr = LogisticRegression(  
    max_iter=2000,  
    class_weight="balanced"  
)
```

```
lr.fit(X_train_vec, y_train)  
pred = lr.predict(X_test_vec)
```

Why this model? It is fast, interpretable, and a strong baseline for sparse text features. It can also provide class probabilities through `predict_proba()`.

10. Train Model 2 — Linear SVM

```
from sklearn.svm import LinearSVC
```

```
svm = LinearSVC(  
    C=1.0,  
    class_weight="balanced"  
)
```

```
svm.fit(X_train_vec, y_train)  
svm_pred = svm.predict(X_test_vec)
```

Linear SVM is often strong for high-dimensional sparse text. Unlike Logistic Regression, LinearSVC does not directly provide calibrated probabilities. Do not call its decision score a probability unless you add calibration.

11. Evaluate the Models

```
from sklearn.metrics import (  
    accuracy_score,  
    precision_recall_fscore_support,  
    classification_report  
)
```

```
print("Accuracy:", accuracy_score(y_test, pred))  
print(classification_report(y_test, pred, digits=4))
```

```
p, r, f1, support = precision_recall_fscore_support(  
    y_test, pred, average="macro", zero_division=0  
)
```

```
print("Macro F1:", f1)
```

- Accuracy.
- Macro precision.
- Macro recall.
- Macro F1.
- Weighted F1.
- Per-class precision/recall/F1.
- Confusion matrix.

12. Confusion Matrix

```
import matplotlib.pyplot as plt  
from sklearn.metrics import ConfusionMatrixDisplay
```

```
ConfusionMatrixDisplay.from_predictions(  
    y_test, pred, xticks_rotation=45  
)  
plt.tight_layout()  
plt.savefig(  
    "reports/figures/confusion_matrix_lr.png",  
    dpi=200  
)  
plt.show()
```

Use the confusion matrix in the FYP to explain which classes are difficult to distinguish. This is more informative than accuracy alone.

13. Hyperparameter Tuning

Run controlled experiments rather than randomly changing parameters. Start with a small search space to keep local training practical.

```
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
```

```
pipeline = Pipeline([
    ("tfidf", TfidfVectorizer()),
    ("clf", LogisticRegression(
        max_iter=2000,
        class_weight="balanced"
    ))
])

params = {
    "tfidf__ngram_range": [(1, 1), (1, 2)],
    "tfidf__min_df": [1, 2, 5],
    "clf__C": [0.5, 1.0, 2.0]
}
```

```
grid = GridSearchCV(
    pipeline,
    params,
    cv=3,
    scoring="f1_macro",
    n_jobs=-1
)
```

```
grid.fit(X_train, y_train)
```

```
print(grid.best_params_)
print(grid.best_score_)
```

Select the configuration using cross-validation on training data. Only after the selection is frozen should you evaluate on the untouched test set.

14. Save the Final Model

The safest deployment pattern is to serialize the complete pipeline so training and inference use exactly the same TF-IDF configuration.

```
import joblib
joblib.dump(grid.best_estimator_, "models/emotion_classifier.joblib")
```

15. Test Local Inference

```
import joblib
```

```
model = joblib.load("models/emotion_classifier.joblib")
```

```
examples = [
    "I finally finished my project and feel amazing.",
    "I wasted the whole day and regret it.",
    "I am nervous about tomorrow."
]
```

```
for text in examples:
    print(text, ">=", model.predict([text])[0])
```

16. Build the FastAPI Model Service

Create app/main.py:

```
from fastapi import FastAPI
from pydantic import BaseModel
import joblib
```

```
app = FastAPI(title="Mentor AI Emotion Classifier")
model = joblib.load("models/emotion_classifier.joblib")
```

```
class PredictionRequest(BaseModel):
    text: str
```

```
@app.post("/predict")
def predict(req: PredictionRequest):
```

```
label = model.predict([req.text])[0]
result = {"emotion": str(label)}

if hasattr(model, "predict_proba"):
    probs = model.predict_proba([req.text])[0]
    result["confidence"] = float(max(probs))

return result
uvicorn app.main:app --reload --port 8000
```

FastAPI provides a local HTTP endpoint. Its interactive API documentation can be used to test the service before connecting Node.js.

17. Connect Node.js to the Python Service

```
const response = await fetch(
  "http://127.0.0.1:8000/predict",
  {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify({text: userMessage})
  }
);
```

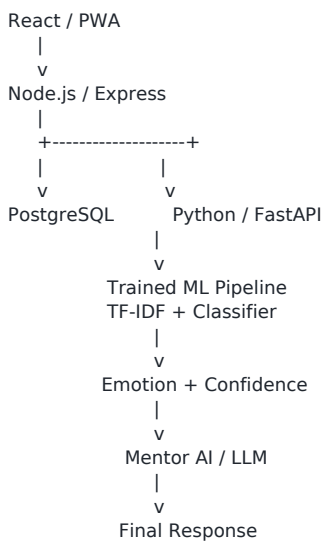
```
const emotion = await response.json();
```

```
// Example:
```

```
// { emotion: "guilt", confidence: 0.84 }
```

Pass the resulting emotion into the existing Mentor AI prompt/context. The classifier should provide structured context; your existing generative AI layer should remain responsible for producing the final mentoring response.

18. Recommended Architecture



19. Experiment Log

ID	Model	Settings	Metric	Purpose
E1	Logistic Regression	TF-IDF (1,2), C=1	Actual result	Baseline
E2	Linear SVM	TF-IDF (1,2), C=1	Actual result	Comparison
E3	Best candidate	Tuned settings	Actual result	Final model

Always enter the actual measured results. Never invent an accuracy or F1 score before running the experiment.

20. Common Mistakes

Data leakage: Fitting TF-IDF on all records before testing.

Test-set tuning: Repeatedly changing the model after looking at test results.

Over-cleaning: Removing negations, emojis, punctuation, or useful informal language.

Class imbalance: Reporting only accuracy when class frequencies differ substantially.

Fake confidence: Treating an SVM decision score as a calibrated probability.

Overengineering: Starting with a transformer before the classical pipeline works.

Deployment mismatch: Using different preprocessing during inference.

Untracked experiments: Changing settings without recording what changed and why.

21. Exact Training Order

- Set up Python environment.
- Create project structure.

- Load and inspect dataset.
- Define exact emotion classes and mapping.
- Clean and prepare data.
- Freeze train/validation/test split.
- Train Logistic Regression baseline.
- Evaluate and save metrics/plots.
- Train Linear SVM.
- Compare macro F1 and per-class results.
- Run limited hyperparameter tuning.
- Evaluate final model once on the untouched test set.
- Serialize the final pipeline.
- Build FastAPI /predict endpoint.
- Connect Node.js.
- Add emotion context to Mentor AI.
- Document actual results, limitations, and future work.

22. Optional Advanced Phase — Transformer

After the classical pipeline works, you may optionally experiment with a small transformer such as DistilBERT. A 6 GB GPU can potentially support a carefully configured fine-tuning run, but it requires more memory management and training time. Treat it as an optional comparison, not a dependency for the core FYP.

23. Final Definition of Done

- Dataset and label mapping documented.
- Training is reproducible.
- At least two models compared.
- Test set protected until final evaluation.
- Metrics and confusion matrix saved.
- Final model serialized.
- Local prediction API working.
- Node.js integration working.
- Mentor AI receives emotion context.
- FYP contains actual measured results and limitations.

Bottom line: Your Lenovo LOQ is fully capable of training the proposed core model locally. The most important part is not GPU power; it is a clean dataset, correct label mapping, leakage-free evaluation, reproducible experiments, and reliable integration.